

ECE264-003 Fall 2019 Exam 1
8-9 PM, Thursday, Sep 19th, 2019

This page is blank.

1. Pointer

```
1.  #include <stdio.h>
2.  #include <stdlib.h>
3.
4.  int f1(int x, int y)
5.  {
6.      int rtv;
7.      int * ptr = & y;
8.      * ptr = * ptr + 5;
9.      rtv = x - y;
10.     return rtv;
11. }
12.
13. int f2(int * x, int * y)
14. {
15.     int * z;
16.     int rtv;
17.     z = x;
18.     x = y;
19.     y = z;
20.     rtv = (* x) - (* y);
21.     return rtv;
22. }
23.
24. void f3(int * x, int * y)
25. {
26.     int z = 16;
27.     * y = * y + 2;
28.     x = &z;
29. }
30.
31. int main(int argc, char ** argv)
32. {
33.     int x = 15;
34.     int y = 3;
35.
```

```
36.    int outA = f1(x, y);
37.    // 1.A: what is the output?
38.    printf("Answer A, value of outA = %d\n", outA);
39.    // 1.B: what is the output?
40.    printf("Answer B, value of y = %d\n", y);
41.
42.    int outC = f2(&x, &y);
43.    // 1.C: what is the output?
44.    printf("Answer C, value of outC = %d\n", outC);
45.    // 1.D: what is the output?
46.    printf("Answer D, value of y = %d\n", y);
47.
48.    f3(&x, &y);
49.    // 1.E: what is the output?
50.    printf("Answer E, value of x = %d\n", x);
51.    // 1.F: what is the output?
52.    printf("Answer F, value of y = %d\n", y);
53.
54.    return EXIT_SUCCESS;
55. }
```

Answer:

1.A: 7

1.B: 3

1.C: -12

1.D: 3

1.E: 15

1.F: 5

2. Call Stack

```
1.  #include <stdio.h>
2.  #include <stdlib.h>
3.  #include <string.h>
4.
5.  char f1(int x, int y)
6.  {
7.      char c = x - y + 'a';
8.      return c;
9.  }
10.
11. int main(int argc, char * * argv)
12. {
13.     if (argc < 3)
14.     {
15.         return EXIT_FAILURE;
16.     }
17.
18.     // 2.A: what is the output?
19.     printf("%ld\n", strlen(argv[1]));
20.
21.     char outB = * argv[1];
22.     // 2.B: what is the output?
23.     printf("%c\n", outB);
24.
25.     // 2.C: what is the output?
26.     char outC = f1(argv[1][1] - argv[2][0], argc);
27.     printf("%d\n", outC);
28.
29.     return EXIT_SUCCESS;
30. }
```

The program's name is q2prog.

The program is called in a terminal using

`./q2prog ECE 264 exam`

Question 2.A - 2.C: please find the program output as program finishes.

Question 2.D - 2.F: please fill the following call stack as the program runs after line 7, before line 8. In the call stack, '-' means do not worry about it.

Call Stack

Frame	Symbol	Address	Value
f1	c	A202	2.D
	y	A201	-
	x	A200	-
	Value Address		2.E
	Return Location		2.F
main	outC	A103	-
	outB	A102	-
	argv	A101	-
	argc	A100	-

Answer:

2.A: 3

2.B: E

2.C: 110

2.D: 'n'

2.E: A103

2.F: line 27

3. Makefile

Consider the following makefile:

```
1.  WARNING = -Wall -Wshadow --pedantic
2.  ERROR = -Wvla -Werror
3.  GCC = gcc -std=c99 -g $(WARNING) $(ERROR)
4.  TESTFALGS = -DTEST_FUNC
5.
6.  SRCS = main.c compare.c testgen.c
7.  # 3.A: fix the command to define OBJS. Please
    include SRCS in the command. Specifically, you
    won't get points for answers like main.o compare.o
    testgen.o
8.  OBJS = $ 3.A
9.
10. q3prog: $(OBJS)
11. $(GCC) $(TESTFALGS) $(OBJS) -o q3prog
12.
13. # 3.B: fix the following command
14. .c.o:
15. $(GCC) $(TESTFALGS) 3.B $*.c
16.
17. testall: test1 test2
18.
19. test1: q3prog
20. ./q3prog inputs/input1 > output1
21. # 3.C: fix the command to compare (and ignore
    space) output1 and expected/expected1.
22. 3.C output1 expected/expected1
23.
24. test2: q3prog
25. ./q3prog inputs/input2 > output2
26. 3.C output2 expected/expected2
27.
```

```
28. clean: # remove all machine generated files
29. rm -f hw06 *.o output*
```

3.D: Suppose we define the following in the code.

```
# ifdef TEST_FUNC
void func1(void)
{
    ...
}
# endif

# ifdef TEST_PRINT
void func2(void)
{
    ...
}
# endif
```

When we use the above makefile to test the program, which function is tested?

Please write your choice of option in **3.D** in the answer sheet.

- A. func1;
- B. func2;
- C. Both func1 and func2;
- D. None of them.

3.E: Using this makefile, what is the linux command to test the program with all test cases (both input1 and input2)? Please write your answer in **3.E** in the answer sheet.

3.F: Using this makefile, what is the linux command to remove all machine generated files after testing? Please write your answer in **3.F** in the answer sheet.

Answer:

3.A: \$(SRCS:%.c=%.o)

3.B: -c

3.C: diff -w

3.D: A

3.E: make testall

3.F: make clean

4. Structure

```
1. #include <stdio.h>
2. #include <stdlib.h>
3. #pragma pack(1) // tell compiler not to pad any
   space
4. // assume sizeof(char) = 1, sizeof(int) = 4,
5. // sizeof(double) = 8, sizeof(a pointer) = 8
6.
7. #define NUMPOINT 5
8. typedef struct
9. {
10.     int x;
11.     int y;
12. } Point;
13.
14. int main(int argc, char ** argv)
15. {
16.     Point * arr;
17.     int ind;
18.
19.     // create an array of NUMPOINT Point objects
20.     // 4.A: fix the code
21.     arr = malloc( NUMPOINT * 4.A );
22.     if(arr == NULL)
23.     {
24.         return EXIT_FAILURE;
25.     }
26.
27.     // initialize the attributes
28.     for (ind = 0; ind < NUMPOINT; ind ++ )
29.     {
30.         arr[ind].x = ind;
31.         arr[ind].y = ind * ind + 5;
32.     }
33.
```

```

34. Point * pptr = arr;
35. // 4.B: what is the output?
36. printf("Answer B = %d\n", pptr->x);
37.
38. pptr += 2;
39. // 4.C: what is the output?
40. printf("Answer C = %d\n", pptr->y);
41.
42. int * aptr = & arr[1].y;
43. // 4.D: what is the output?
44. printf("Answer D = %d\n", * aptr);
45.
46. aptr += 3;
47. // 4.E: what is the output?
48. printf("Answer E = %d\n", * aptr);
49.
50. // 4.F: If the memory is not released, how many
51. // bytes of memory are leaked?
52. free(arr);
53.
54. return EXIT_SUCCESS;
55.}

```

Answer:

4.A: sizeof(Point) or sizeof(*arr);

4.B: 0

4.C: 9

4.D: 6

4.E: 3

4.F: 40

ASCII Table

Dec	Char	Dec	Char	Dec	Char	Dec	Char
0	NUL	32	SPACE	64	@	96	`
1	SOH	33	!	65	A	97	a
2	STX	34	"	66	B	98	b
3	ETX	35	#	67	C	99	c
4	EOT	36	\$	68	D	100	d
5	ENQ	37	%	69	E	101	e
6	ACK	38	&	70	F	102	f
7	BEL	39	'	71	G	103	g
8	BS	40	(	72	H	104	h
9	TAB	41	)	73	I	105	i
10	LF	42	*	74	J	106	j
11	VT	43	+	75	K	107	k
12	FF	44	,	76	L	108	l
13	CR	45	-	77	M	109	m
14	SO	46	.	78	N	110	n
15	SI	47	/	79	O	111	o
16	DLE	48	0	80	P	112	p
17	DC1	49	1	81	Q	113	q
18	DC2	50	2	82	R	114	r
19	DC3	51	3	83	S	115	s
20	DC4	52	4	84	T	116	t
21	NAK	53	5	85	U	117	u
22	SYN	54	6	86	V	118	v
23	ETB	55	7	87	W	119	w
24	CAN	56	8	88	X	120	x
25	EM	57	9	89	Y	121	y
26	SUB	58	:	90	Z	122	z
27	ESC	59	;	91	[	123	{
28	FS	60	<	92	\	124	
29	GS	61	=	93	]	125	}
30	RS	62	>	94	^	126	~
31	US	63	?	95	_	127	DEL